

Lecture 1 : Motivation

ESU 207

Data Structures & Algorithms

* Most of the content from Surender Baswana's
lecture notes (a big thanks)

Logistics

- 1) Instructor
- 2) Tutors & TA
- 3) Communication
 - ↳ Hello
 - ↳ Prutor
 - ↳ Gradescope
- 5) Evaluation & Grading
- 6) Cheating policy

Important

1) Attendance (2%)

10th - 14th
(No class)

2) Extra Class

Mon 2-4 pm
(3rd August)

L20

3)

Mon (2-5 pm)
Wed

→ 1st Nov (Sunday)

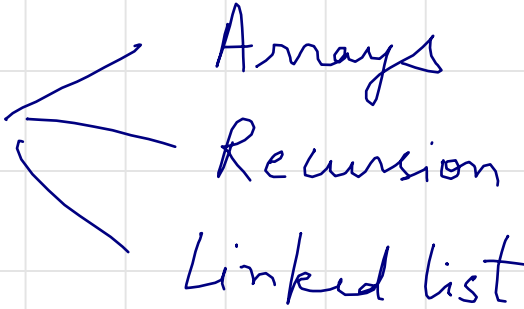
4)

Cheating

5)

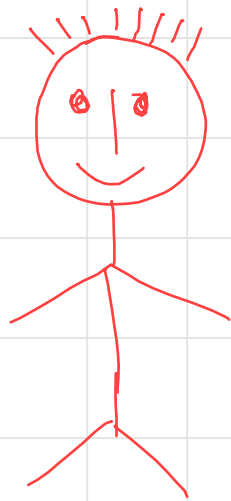
Prog. Language

Background

- 1) Basics of Programming 
 - Arrays
 - Recursion
 - Linked list
- 2) Interest in puzzles/math.
- 3) Philosophy / lesson / idea (keep a watch)

Story

You



Give me answer



Here it is

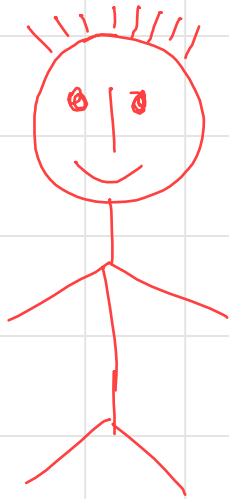


Laloo Lal
Man



Story

You



Give me answer



Here it is



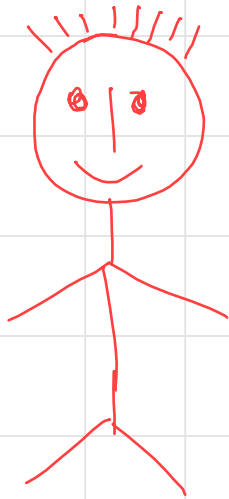
Laloo Lal
Man



— Powerful

Story

You



Give me answer



Here it is



Laloo Lal
Man

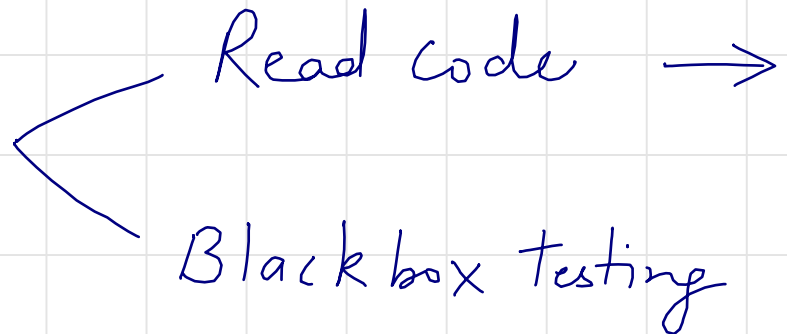


- Powerful
- Can't trust

Q. What should
we do with
LLM?

Big Question

Q. What should we do with LLM?

— Verify code  Read code → Write Code
Black box testing

— Proof of correctness

* Use it as a mentor, not to cheat

Today's lecture

- 1) Overview
- 2) Motivation
- 3) A few examples

Main Objective:

Solving problems on **computer**

Philosophy:

Discover the solution instead of
reading it

Active in class

Active in HW/Assignments

Main Objective:

Solving problems on **computer**

Philosophy:

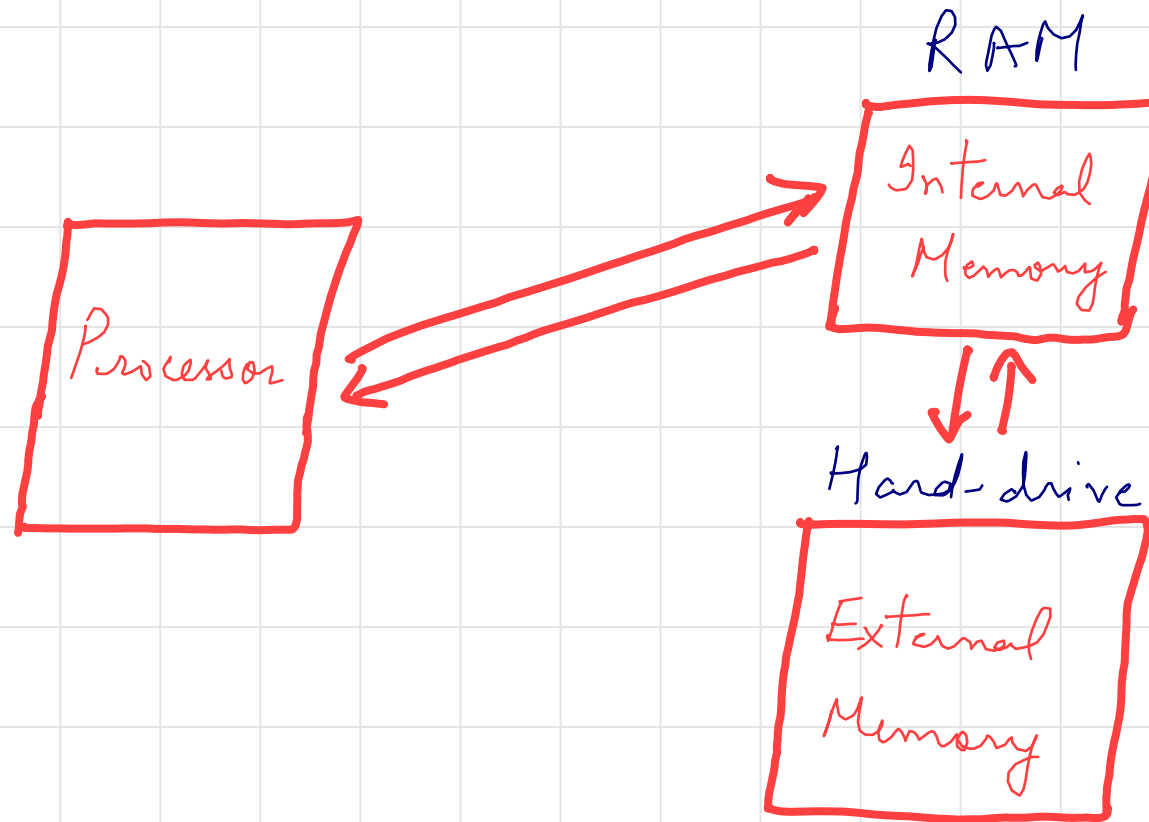
Discover the solution instead of
reading it

Active in class

Active in HW/Assignments

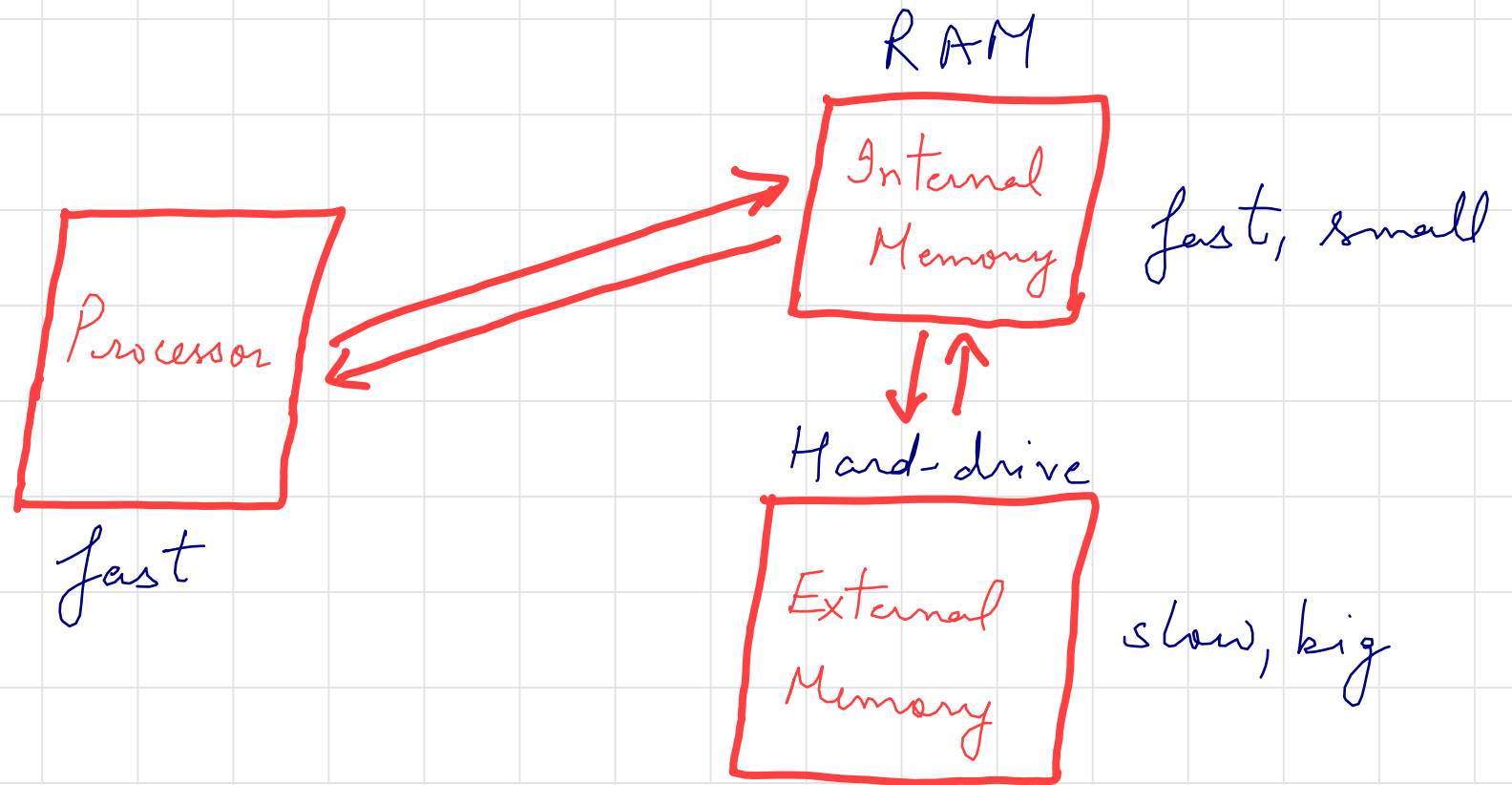
* Do not tell answer if you already know it!

Schema of a computer



* Refine it further

Schema of a computer



Assumptions

- word RAM model:
 - We need constant time to execute an instruction
- You understand programming
 - focus on problem solving
 - ignore syntax/nitty-gritty of programming

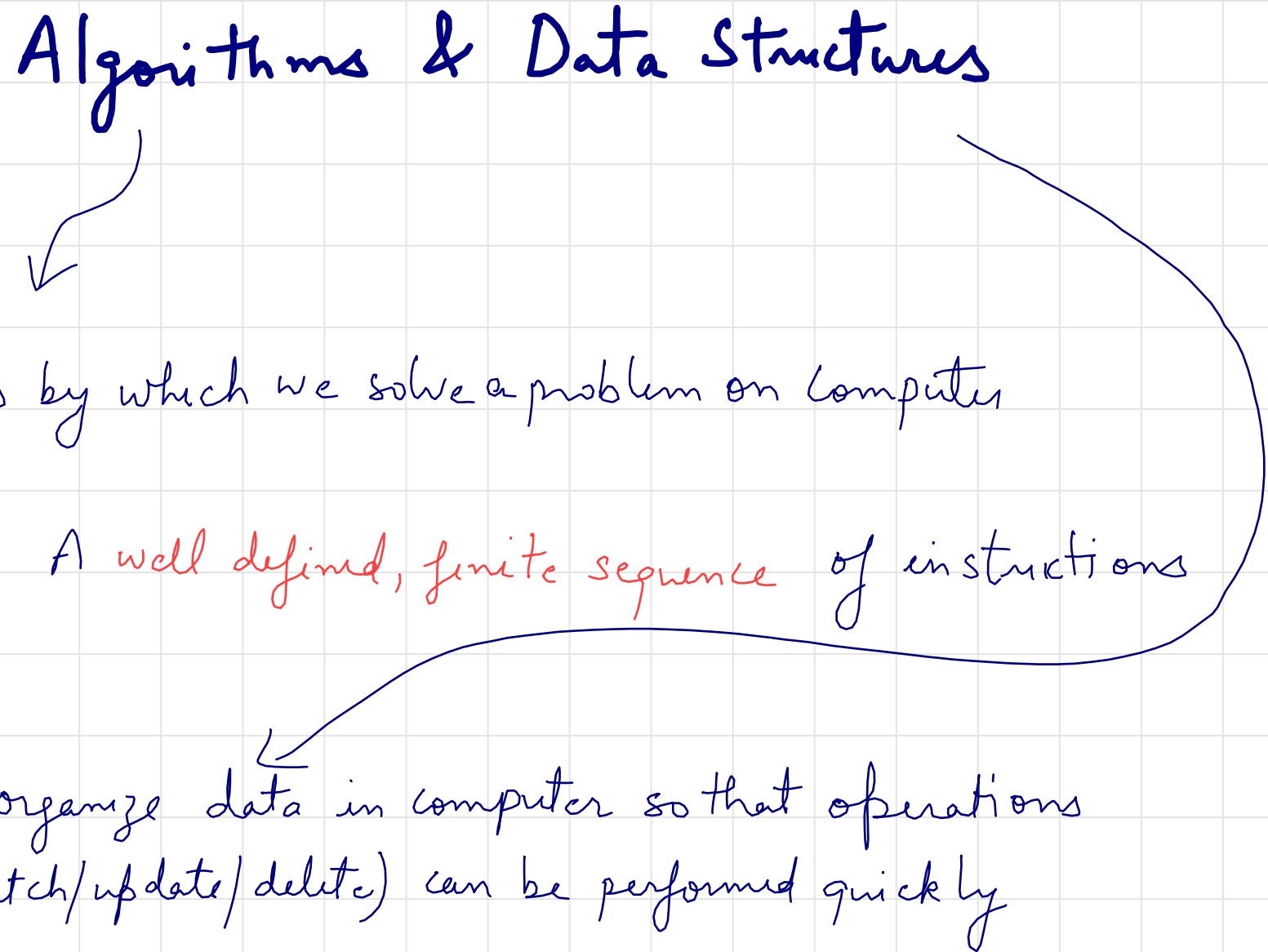
Algorithms & Data Structures



The process by which we solve a problem on computer

Formally → A well defined, finite sequence of instructions

Algorithms & Data Structures



The process by which we solve a problem on computer

Formally → A well defined, finite sequence of instructions

Aim: Store/organize data in computer so that operations on it (fetch/update/delete) can be performed quickly

Algorithms & Data Structures



The process by which we solve a problem on computer

Formally → A well defined, finite sequence of instructions



Aim: Store/organize data in computer so that operations on it (fetch/update/delete) can be performed quickly

Learn: Efficient algorithms & data structures

Algorithms

Philosophy: Take a look at trivial algos, can we make them fast (efficient)

Problem 1: Compute $x^n \bmod m$, say $m = 2025$
 $x, n, m \rightarrow \text{big}$

Trivial algo:

Algorithms

Philosophy: Take a look at trivial algos, can we make them fast (efficient)

Problem 1: Compute $x^n \bmod m$, say $m = 2025$
 $x, n, m \rightarrow \text{big}$

Trivial algo: Recursion $x^n = x \times x^{n-1}$
then compute mod

Algorithms

Philosophy: Take a look at trivial algos, can we make them **fast (efficient)**

Problem 1: Compute $x^n \bmod m$, say $m = 2025$
 $x, n, m \rightarrow \text{big}$

Trivial algo: Recursion

$x^n = x \times x^{n-1}$ $\rightarrow$ Wasteful
then compute **mod** $\rightarrow$ last step?

Trivial algo:

Recursion

$$x^n = x \times x^{n-1}$$

then compute

mod

last step?

→ Wasteful

Better algo:

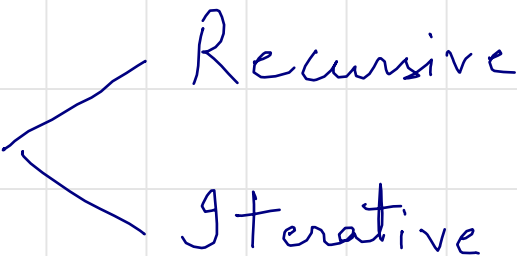
$$\begin{array}{l} x^n \\ \swarrow \quad \searrow \\ x^{n/2} \quad x^{n/2} \quad \text{if } n=1 \\ \quad \quad \quad \times \quad \quad \quad \text{n even} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ x^{n/2} \quad x^{n/2} \quad x^{n/2} \quad x^{n/2} \quad \text{n odd} \end{array}$$

- * Also, compute mod at every step
- * Try it at home
- * Why is it correct?

Problem 2 Fibonacci Numbers

$$F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)$$

* Show $F(n)$ is big $> 2^{(n-2)/2}$ (Induction)

Two algorithms 
Recursive
Iterative

Fibonacci

RFib

{

```
if (n==0) return 0;  
else if (n==1) return 1;  
else return (Rfib(n-1)+Rfib(n-2))
```

}

Fibonacci

RFib

```
{
  if (n==0) return 0;
  else if (n==1) return 1;
  else return (Rfib(n-1)+Rfib(n-2));
}
```

Ifib

```
{
  if (n==0) return 0
  else if (n==1) return 1
  else {
    a ← 0 ; b ← 1;
    for i=2 to n do
      { temp ← b;
        b ← a+b;
        a ← temp;
      }
    return b;
  }
}
```


Which one is better?

Is it correct?
→

RFib

```
{
  if (n==0) return 0;
  else if (n==1) return 1;
  else return (Rfib(n-1)+Rfib(n-2));
}
```

IFib

```
{
  if (n==0) return 0
  else if (n==1) return 1
  else {
    a ← 0 ; b ← 1;
    for i=2 to n do
      { temp ← b;
        b ← a+b;
        a ← temp;
      }
    return b;
  }
}
```

HW: Coding

1) $x^n \bmod 2025$

2) Rfib & Ifib for $F(n) \bmod 2025$

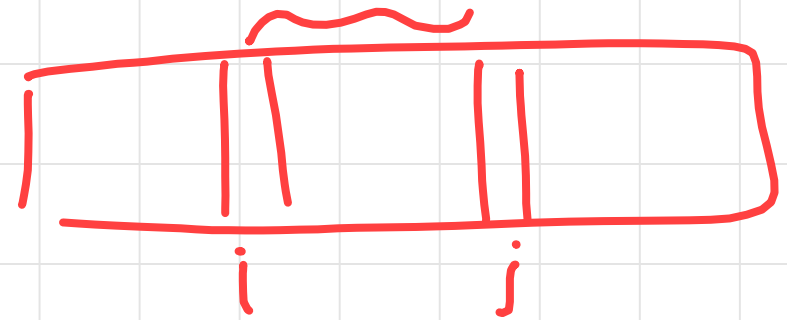
up to what values of n can you compute
in 10sec, 1 min, 10 mins.

Lesson: We need efficient algorithms in real world (Google, LLM)

Example: Data structures

$A[i \dots j]$

Range Minima Problem



Input: an array A storing n numbers

Query: i, j (can be done multiple times)

Output (of query): $\min(A[i], A[i+1], \dots, A[j])$

- String matching
- Computational geometry
- Subroutine for various algorithms

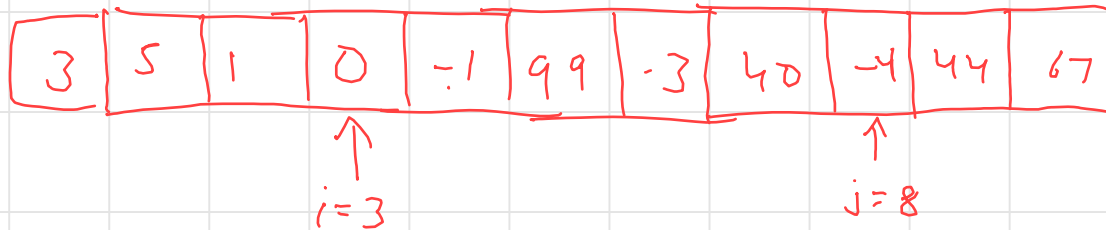
Example: Data structures

Range Minima Problem

Input: an array A storing n numbers

Query: i, j (can be done multiple times)

Output (of query): $\min(A[i], A[i+1], \dots, A[j])$



$$RM(3, 8) = -4$$

$n \sim 1$ million

of queries ~ 10 million

Trivial algorithm

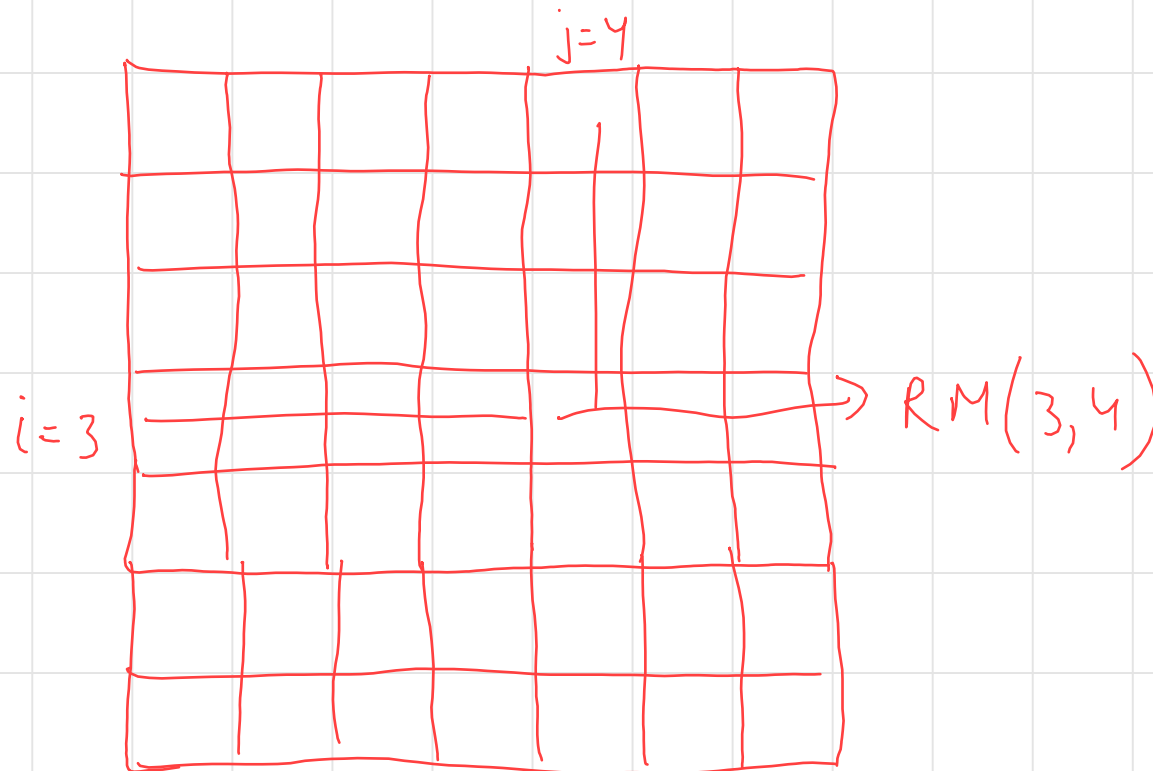
```
{ temp ← i+1  
  min ← A[i]  
  while [temp ≤ j]  
  { if (min > A[temp])  
    min ← A[temp];  
    temp ← temp + 1;  
  }  
  return min  
}
```

* Good for 1 query, but # of ~ 10 million

Another approach - almost trivial

* Store $RM(i, j)$ (all $\sim n^2$ entries) in an array

$RM =$

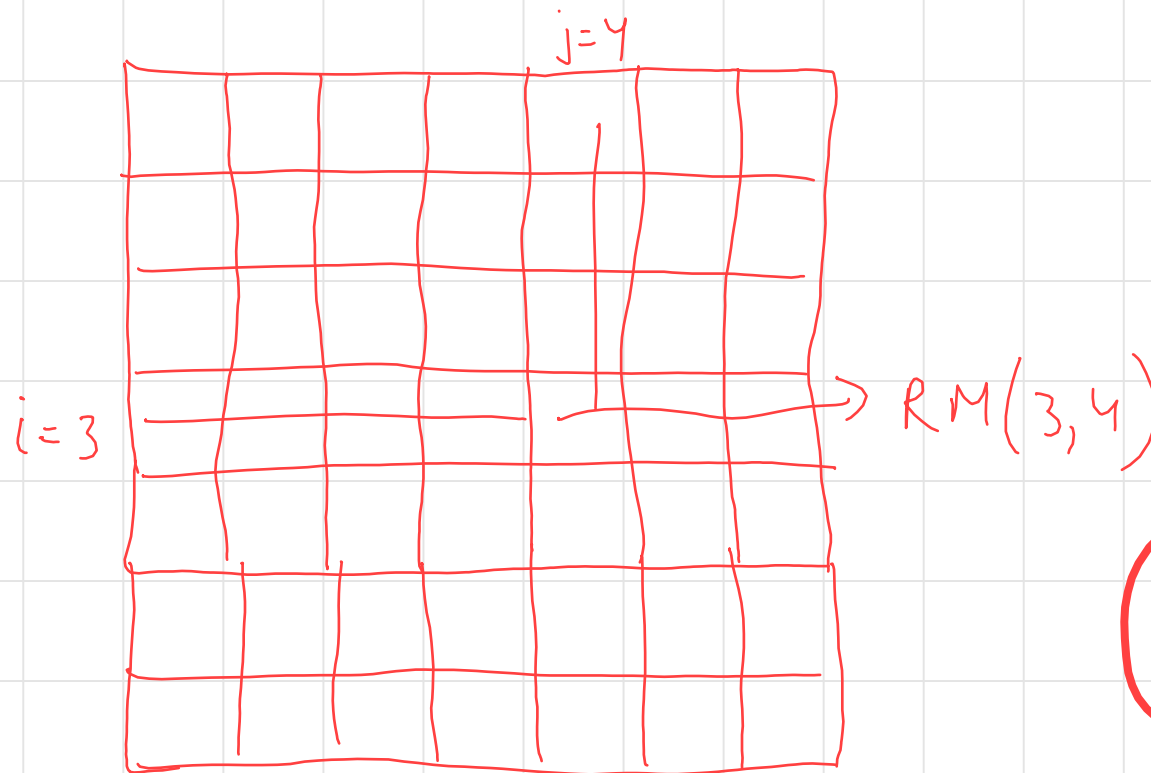


* why bad.

Another approach - almost trivial

* Store $RM(i, j)$ (all $\sim n^2$ entries) in an array

$RM =$



* why
bad.

Too
much
space

Can we find a compact & efficient solution
for Range-Minima, Proof of correctness

Try, will do it later.

- HW
- Recursion